

DRF-ECCT: Dynamic Reliability Filter for Error Correction Code Transformer

1st Dang-Trac Nguyen

Department of Electrical,
Electronic and Computer Engineering
University of Ulsan,
Republic of Korea
tracng417@gmail.com

2nd Sunghwan Kim

Department of Electrical,
Electronic and Computer Engineering
University of Ulsan,
Republic of Korea
sungkim@ulsan.ac.kr

Abstract—In this work, we propose dynamic reliability filter (DRF) mechanism to enhance the code-aware self-attention of error correction code transformer (ECCT). The DRF is designed to dynamically filter out the low magnitudes positions in the query and key matrices, amplifying the reliable information. By this mechanism, the code-aware self-attention of the ECCT is better utilized, effectively extracting the noise corrupted positions from the channel output. Additionally, layer reuse is introduced for the feed-forward sub-layers to increase model efficiency, enabling the use of deeper network with less parameters compared to the baseline ECCT. The experimental results show considerable improvements in bit error rates compared to the baseline ECCT, while also significantly increasing the convergence speed during the training stage.

Index Terms—Transformer, error correction code transformer (ECCT)

I. INTRODUCTION

Among the technologies utilized in wireless communication systems, error correction codes (ECCs) is one of the essential factors contributing to the reliability of data transmission. ECCs are known for their capability of correcting errors in the corrupted received signals by relying on complex likelihood decoding algorithms such as belief propagation (BP) decoding. Despite its advantages, the NP-hard level of complexity in the decoding process have also challenged the advancement of ECCs.

In recent years, the learning-based ECC decoding have gained favor due to their superior performance compared to traditional methods. The different approaches range widely from model-free designs [1]–[4] to model-based adaptation of traditional decoding algorithm with neural networks [5]–[9]. While the model-based designs adopting the BP algorithm suffer from the complexity of the Tanner-graphs, the model-free approaches, though less restrictive, are hindered by the curse of dimensionality, especially when dealing with significantly large code length.

As an alternative to direct codeword estimation, a neural network with model-free syndrome-based decoding [12] focused

This work was supported by the Research Program through the National Research Foundation of Korea under Grant NRF-2022M3C1A3081366 and NRF-2023R1A2C1003546.

on training the network to estimate the multiplicative noise from the channel output and removing the noise to obtain the approximated transmitted signal. By adopting this principle, the authors of [10] proposed the transformer-based method, namely error correction codes transformer (ECCT), that successfully adapted the components of the original transformer in [11] to decoding ECCs. The ECCT in [11] was able to achieve a significant reduction in error probability compared to the state-of-the-art BP-based neural decoding approaches as well as other previous learning-based models including that of [12]. Nevertheless, this approach was the first attempt at transformer-based decoders, which had not fully explored potential of transformer-based decoder through the reliability decoding.

As an attempt to explore the compatibility of transformer and reliability decoding as well as improving the performance of ECCT [10], this work proposes a dynamic reliability filter mechanism for ECCT. The dynamic reliability filter (DRF) mechanism is proposed. It takes as input either the query or key matrices to produced a concentrated version of its input. The DRF operates using sets of adaptive threshold and trainable parameters uniquely defined for each attention head. To effectively reduce the number of trainable parameters, layer reuse is introduced to the ECCT architecture through moderate reuse of the transformer layers. This form of layer reuse reduce approximately 22% of the overall model size compared to the baseline ECCT. From experimentation, the proposed methods achieves considerable improvements in increasing the convergence speed during training as well as reducing bit error rates (BER) during inference stage, resulting in improvements between 0.2dB and 0.4dB. All models are verified on Bose-Chaudhuri-Hocquenghem (BCH) code, BCH(63,51), low-density parity-check (LDPC) code, LDPC(121,80), and Polar code, Polar(64,32).

The rest of this paper is organized as follows. Section II introduces the related knowledge to this work. Section III consists of detailed explanations for proposed methodology. The experimental results and discussions are described in Section IV. Finally, we provide our conclusions to this work in Section V.

II. BACKGROUND AND RELATED WORKS

In this section, necessary background knowledge is provided for coding, transformer, and key features of the ECCT [10].

A. Coding

Assuming a standard wireless communication system, transmission is conducted with a linear code C to encode an input message $\mathbf{m} \in \{0,1\}^k$ into a codeword $\mathbf{x} \in \{0,1\}^n$ through the Galois field matrix product with a generator matrix $\mathbf{G}$ (summation operation is replaced by the modulo-2 operation). The linear code is defined by a binary generator matrix $\mathbf{G}$ of size $k \times n$ and a binary parity-check matrix $\mathbf{H}$ of size $(n-k) \times n$, such that $\mathbf{GH}^T = 0$, where T denotes the matrix transposition. The codeword $\mathbf{x}$, satisfying $\mathbf{Hx} = 0$, is modulated into $\mathbf{x}_s$ and transmitted via a binary-input-symmetric-output, additive white gaussian noise (AWGN) channel. Without loss of generality, the modulated codeword $\mathbf{x}_s$ (commonly considered binary phase shift keying (BPSK)) can be obtained by $\mathbf{x}_s = \text{bipolar}(\mathbf{x})$, where $\text{bipolar}(\cdot)$ denotes the conversion by $0 \rightarrow 1, 1 \rightarrow -1$, and is also obtainable as $\text{bipolar}(a) = 1 - 2a$ with input $a \in \{0,1\}$. The effect of the channel can be modeled as $\mathbf{y} = \mathbf{x}_s + \mathbf{z}$, where $\mathbf{z}$, defined by $\mathbf{z} \sim \mathcal{N}(0, \sigma^2)$, represents the AWGN noise vector which is independent of the codeword $\mathbf{x}$. At the decoder, the main objective is to provide a soft estimation $\hat{\mathbf{x}}$ of the original codeword. Thus, the decoding function can be viewed as a function of $\mathbf{y}$, which is $\hat{\mathbf{x}} = f(\mathbf{y})$.

B. Transformer

Originally, the transformer was introduced in [11] as a successor to RNN in sequence-to-sequence learning tasks. Since then, the transformer gains popularity in variety of fields [14], [15], including communication tasks [16], [17]. The vanilla transformer in [11] consists of two major components: the positional encoding and multi-head attention mechanism. At the heart of the transformer, and also the key to its success, is the multi-head self-attention (MHSA) mechanism. Each head of the MHSA operates as a scaled dot-product attention, which is defined as

$$A(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_p}}\right)V, \quad (1)$$

where d_p is the dimension of the multi-head parallel linear projections of query and key vectors, which is calculated by d_{model}/h in which d_{model} denotes the embedding dimension after applying positional encoding; h denotes the number of parallel heads. The MHSA is formulated as $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)\Theta^O$, where $\text{head}_i = A(Q\Theta_i^Q, K\Theta_i^K, V\Theta_i^V)$ is the scaled-dot product attention for each head, in which $\Theta_i^Q, \Theta_i^K, \Theta_i^V \in \mathbb{R}^{d_{\text{model}} \times d_p}$, and $\Theta_i^O \in \mathbb{R}^{hd_p \times d_{\text{model}}}$ are input and output parameter matrices. In [11], an modification of the MHSA, called masked self-attention, taking the summation of causal masks and the dot-product of the query-key pair to prevent the decoder from attending to subsequent positions in the same input sequence.

C. Error Correction Code Transformer

The authors of [10] introduced two key features, namely positional reliability encoding and code-aware self-attention, which will be discussed in this subsection.

1) *Positional reliability encoding*: Each dimension of $\{\tilde{\mathbf{y}}_i\}_{i=1}^{2n-k}$ in the positional encoding is regarded as the pre-processed version of the channel output $\mathbf{y}$ by (3). The encoding process projects the vector $\tilde{\mathbf{y}}$ to a d_{model} dimensional embedding $\{\delta_j\}_{j=1}^{2n-k}$, which is defined by

$$\delta_j = \begin{cases} |\mathbf{y}_j| \Theta_j, & \text{if } j \leq n, \\ (1 - 2(s(\mathbf{y}))_{j-n+1}) \Theta_j, & \text{otherwise,} \end{cases} \quad (2)$$

where $\{\Theta_j \in \mathbb{R}^{d_{\text{model}}}\}_{j=1}^{2n-k}$ denotes the parameterized encoding vector corresponding to each bit position of $\tilde{\mathbf{y}}$. As was shown in [10], this embedding method yields a desirable effect during the self-attention dot product operation such that the unreliable information (i.e., negative-valued positions) collapse to the origin, whereas the syndrome information yield negative scaling.

2) *Code-aware self-attention*: The ECCT incorporated unique sparse masks, capturing the pattern of the parity-check matrices corresponding to different code structures. For a given algebraic code corresponding to a parity-check matrix $\mathbf{H}$, the attention mask is denoted by $g(\mathbf{H}) : \{0,1\}^{(n-k) \times n} \rightarrow \{-\infty, 0\}^{(2n-k) \times (2n-k)}$. The construction of the mask $g(\mathbf{H})$ is explained by Algorithm 1.

Algorithm 1 Pseudo-code of the binary mask generating algorithm

Input: Parity-check matrix $\mathbf{H}$

Output: Mask based on $\mathbf{H}$

$\mathbf{M} \leftarrow I(2n-k)$

for $i = 1, 2, \dots, n-k$ **do**

$idx \leftarrow \text{where}(\mathbf{H}[i] == 1)$

for $j \in idx$ **do**

$\mathbf{M}[n+i, j] \leftarrow 1$

$\mathbf{M}[j, n+i] \leftarrow 1$

for $h \in idx$ **do**

$\mathbf{M}[j, h] \leftarrow 1$

$\mathbf{M}[h, j] \leftarrow 1$

end

end

end

$\mathbf{M} \leftarrow (-\infty(\neg \mathbf{M}))$

III. PROPOSED ERROR CORRECTION CODE TRANSFORMER

In this section, the proposed ECCT is discussed in detail. The overall system model is presented, including the pre-processing and post-processing of data prior to the neural network decoder. The proposed methods, including the DRF mechanism and layer reuse, are thoroughly explained.

A. System model

In Fig. 1, the overall system model for the communication system is presented, including the encoding, modulation, and decoding processes. Similar to [10], at the decoder, the pre-processing and post-processing methods proposed by [12] are considered in this work. These methods are design on the basis of syndrome-based decoding, aiming to overcome overfitting and eliminate the need for the simulation of codewords during training.

The pre-processing step modifies channel output $\mathbf{y}$ with a vector of length $2n - k$, which can be defined as

$$\tilde{\mathbf{y}} = C(|\mathbf{y}|, s(\mathbf{y})), \quad (3)$$

where $C(\cdot)$ denotes the vector concatenation, $|\mathbf{y}|$ represents the magnitude for individual element in $\mathbf{y}$, and $s(\mathbf{y}) = \mathbf{H}\mathbf{y}_b \in \{0, 1\}^{n-k}$ denotes the syndrome vector obtained by Galois field matrix product between the hard-decision $\mathbf{y}_b$ of $\mathbf{y}$ and the parity-check matrix $\mathbf{H}$.

The binary hard-decision of $\mathbf{y}$ is defined as $\mathbf{y}_b = \text{bin}(\text{sign}(\mathbf{y})) = \frac{1}{2}(1 - \text{sign}(\mathbf{y}))$, where $\text{bin}(\cdot)$ denotes the inverse of the *bipolar*($\cdot$) operation, and $\text{sign}(\cdot)$ denotes a sign function. An inverse BPSK operation, denoted as $\text{bin}(\cdot)$ is defined by $\text{bin}(b) = (1 - b)/2$, where input $b \in \{\pm 1\}$ is the result of the BPSK bipolar conversion. The sign function, denoted as $\text{sign}(\cdot)$, is defined by $\text{sign}(c) = \begin{cases} 1, & c > 0, \\ 0, & c = 0, \\ -1, & c < 0. \end{cases}$

The post-processing step multiplies the bipolar mapping of the channel output $\mathbf{y}$ with the decoder output to create the estimation $\hat{\mathbf{x}}$ of the original codeword $\mathbf{x}$. The prediction takes the form of $\hat{\mathbf{x}} = \mathbf{y} \otimes f_{\Theta}(\tilde{\mathbf{y}})$, where $f_{\Theta}(\cdot)$ denotes the neural decoder parameterized by Θ and $\otimes$ is the component-wise multiplication.

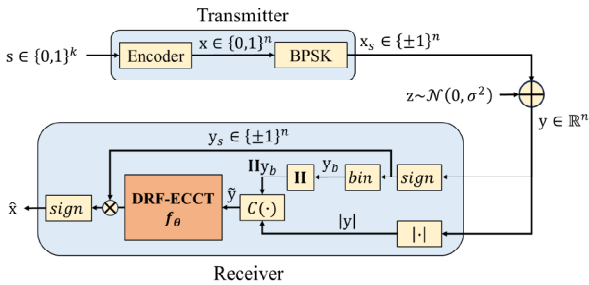


Fig. 1. System model for syndrome-based decoding framework (orange module indicates the proposed components).

B. Dynamic reliability filter - DRF

1) *Approach*: The objective of the neural decoder is the prediction of the multiplicative noise $\tilde{\mathbf{z}}$ that mitigates the additive noise $\mathbf{z}$ from the channel output $\mathbf{y}$. By the formula of $\mathbf{y} = \mathbf{x}_s \tilde{\mathbf{z}}_s$ (proof of Lemma 1 in [13]), the multiplicative noise is obtained by $\tilde{\mathbf{z}}_s = \tilde{\mathbf{z}}_s \mathbf{x}_s^2 = \mathbf{y} \mathbf{x}_s$. Hence, the target binary multiplicative noise is defined by $\tilde{\mathbf{z}}_b = \text{bin}(\text{sign}(\mathbf{y} \mathbf{x}_s))$. Thus,

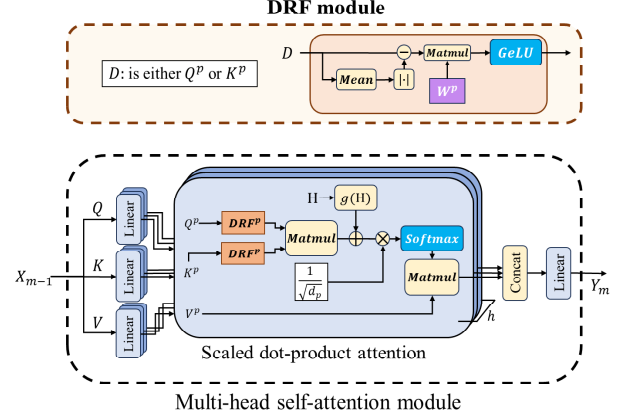


Fig. 2. Diagram of DRF module and its position in the multi-head self-attention sub-layer (orange module indicates the proposed components).

the objective function for training the ECCT, in the form of the binary cross-entropy (BCE) loss function, is formalized as

$$\mathcal{L}_{\text{BCE}} = - \sum_{i=1}^n \tilde{\mathbf{z}}_i \log(f_{\Theta}(\tilde{\mathbf{y}})) + (1 - \tilde{\mathbf{z}}_i \log(1 - f_{\Theta}(\tilde{\mathbf{y}}))). \quad (4)$$

Thus, the estimation of the original codeword $\mathbf{x}$ is obtained by $\hat{\mathbf{x}} = \text{bin}(\text{sign}(f_{\Theta}(\tilde{\mathbf{y}}) \text{sign}(\mathbf{y})))$. Under the mentioned objective, the attention mechanism for high-reliable information (positions with high magnitude in the channel output) becomes very essential for estimating the multiplicative noise $\tilde{\mathbf{z}}$. In light of the given objective function, we provide analysis for the effect of code-aware attention mechanism on the multiplicative estimation of channel noise. Considering the binary mask $g(\mathbf{H})$ generated by Algorithm 1, the masked self-attention mechanism can be defined as

$$A_{\text{mask}}(Q, K, V) = \text{Softmax}\left(\frac{QK^T + g(\mathbf{H})}{\sqrt{d_p}}\right)V. \quad (5)$$

By summation with $g(\mathbf{H})$ mask, the values of desirable positions in the dot-product are retained while the prohibited positions theoretically yield $-\infty$ which collapse to the nearest possible value to zero after the *Softmax* is applied. Hence, the pattern of attention weight matrix follows the symmetrical format of $g(\mathbf{H})$. Through this operation, the attention mechanism can be regarded as a reliability learning module (attention to high magnitude positions). However, the binary design of the current masked self-attention only restricts non-desirable positions, similar to the traditional transformer [11]. Therefore, further improvements for ECCT can be achieved through additional operation that amplify the reliable information in the dot-product.

2) *High-reliability filter mechanism of DRF*: The proposed DRF module is designed to filter the unreliable information and amplify the high magnitude positions in query and key matrices. The filtered query and key matrices contributes to a much more concentrated attention weight matrix after the scaled dot-product operation, thus refine and accelerate the

attention mechanism. For each independent attention head $g \in \{1, \dots, h\}$, DRF learning process is defined by

$$DRF^p = GeLU((D^p - |\tau^p|) \cdot W^p), \quad (6)$$

where $GeLU$ is the activation introduced in [18]. The weight matrices defined by $W^p \in \mathbb{R}^{h \times d_p \times 1}$ are independently initialized and trained for each attention head. D^p indicates the input to the DRF module for individual attention head, which can be either Q^p or K^p . The bias term τ^p is a set of adaptive thresholds unique to each attention head and is defined as the arithmetic mean of the dot product matrix for each head, $\tau^p = \frac{1}{(2n-k)^2} \sum_{i=1}^{2n-k} \sum_{j=1}^{2n-k} D_{i,j}^p$. Hence, by applying DRF to the self-attention mechanism, (1) can be rewritten as

$$A_{DRF}(Q, K, V) = Softmax\left(\frac{DRF(Q)DRF(K)^T + g(\mathbf{H})}{\sqrt{d_p}}\right)V, \quad (7)$$

where DRF is the general output of the overall reliability filter mechanism consisting of DRF^p in (9) for every attention head. As suggested in Fig. 2, the DRF module is reuse for processing both query and key matrices in the same MHSA module. Since the same input is linearly projected to three different mappings Q, K, V matrices in the MHSA module, sharing the same set of weights in the DRF operation for both Q and K matrices provides an efficient and effective way of learning to filter two different projections of the same input.

During analysis of the attention patterns, it is observed that the distribution of the attention weight matrix tends to spread out across the hidden dimension space, thus yielding high arithmetic means. Hence, the attention product is less concentrated, decelerating the learning process. To counter this effect, the subtraction of the absolute value of τ is introduced to negatively affect the mid to low-magnitude positions. In combination with the $GeLU$ activation function [18], τ is considered a threshold below which the input values are negatively mapped to the output or forced to *zero*. Furthermore, by matrix multiplication with the weight W^p , the information in the embedding dimension is collected at a single node, representing only the highest magnitudes. This phenomenon reduces the noise that initially hinders the attention process to reliable information.

C. Layer reuse strategy

As an effort to enhance model efficiency for the DRF-ECCT, layer reuse is introduced. During model size inspection, the FFN sub-layer was found to use a higher number of parameters than the MHSA sub-layer. Therefore, in order to efficiently utilize the FFN sub-layers, only two FFN layers are initialized in an M' -layers architecture, each reused for even-indexed and odd-indexed transformer layers (see Fig. 3). By this mechanism, the total number of parameters used by all M FFN sub-layers is significantly reduced.

For the efficiency enhancement effect of the layer reuse schemes to be fully explained, the quantification of parameters in the ECCT models is required. In the sub-layer architecture of ECCT in [10], three main sub-layer types are used, namely

TABLE I
QUANTIFICATION OF PARAMETER IN SUB-LAYER AND OVERALL ARCHITECTURE.

Modules	Parameter calculation
P_{MHA}	$4P_{dense}(d_{model}, d_{model})$ $= 4d_{model}^2 + 4d_{model}$
P_{FFN}	$P_{dense}(d_{model}, d_{ff}) + P_{dense}(d_{ff}, d_{model})$ $= 8d_{model}^2 + 5d_{model}$
P_{ECCT}	$MP_{MHA} + MP_{FFN} + 2MP_{LN}$ $12Md_{model}^2 + 13Md_{model}$
$P_{DRF-ECCT}$	$(M+4)P_{MHA} + 2P_{FFN} + (M+6)P_{LN}$ $= (4M+32)d_{model}^2 + (5M+32)d_{model}$

layer normalization (LN), multi-head attention layer (MHA) and feed-forward layer (FFN). For ECCT [10], the number of parameters used by each type of sub-layer can be calculated by a function of the embedding dimension d_{model} . The building block of the all sub-layers is called dense layer or fully connected layer with P_{dense} parameters, calculated by

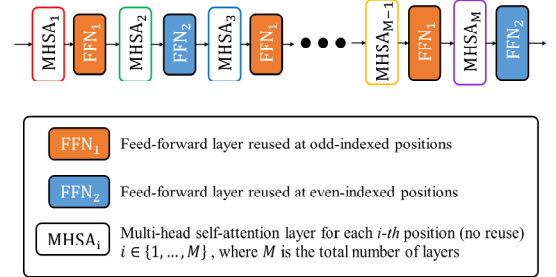


Fig. 3. Layer reuse strategy.

$$P_{dense}(d_{in}, d_{out}) = d_{in} \times d_{out} + d_{out}, \quad (8)$$

where d_{in} and d_{out} are the input and output dimensions of the dense layer respectively. The number of parameters for layer normalization, P_{LN} , is defined as

$$P_{LN} = 2d_{model}. \quad (9)$$

From the fundamental formula in (8) and (9), the number of parameters for MHA, FFN sublayers and the overall model are defined in Tab. I. In this work, a DRF-ECCT architecture with $M' = M+4$ is considered in experimentation, which provides not only parameters reduction but also noticeable performance improvements.

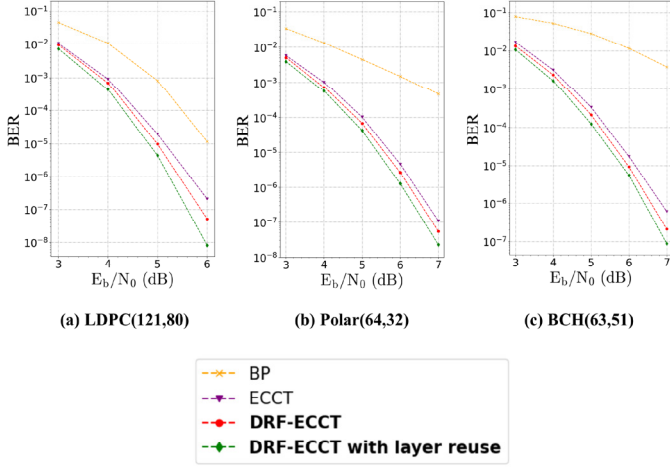


Fig. 4. BER comparison between BP, ECCT [10], and our proposed models.

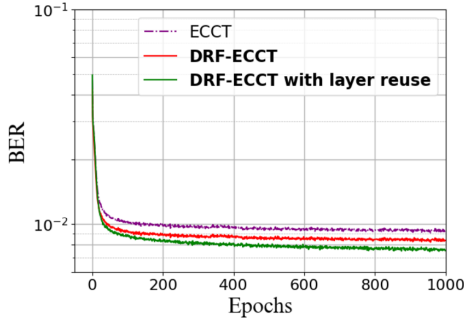


Fig. 5. BER convergence over 1000 epochs for LDPC(121,80).

IV. EXPERIMENTATION

A. Setup and training

The experimentation incorporates the baseline of ECCT [10] and BP [21] with 50 iterations. To retain fairness in comparison, training and inference stages are executed on all neural network models with similar hyper-parameter settings, $h = 8$ attention heads, and embedding dimension $d_{\text{model}} = 128$. As discussed in previous section, models without layer reuse are set with $M = 6$ layers while layer reuse models uses $M' = 10$ layers but only initialized with half that number. Similar to [10], our design is evaluated with three families of codes, namely LDPC codes, Polar codes, and BCH codes. The parity-check matrices for all codes are taken from [19].

For training, data samples are generated with 128 samples per batch for 1000 batches, accumulates to 128000 samples, which are all-zero codewords similar to [10], [12]. All neural network models are trained on noise-corrupted codewords with the energy per bit to noise power spectral density ratio E_b/N_0 within the range of 2dB and 8dB. The Adam optimizer [20] is used for training with initial learning rate of 10^{-4} , over 1000 epochs, and with cosine annealing weight decay method [22] (without warm restart) at minimum the learning rate of 5×10^{-7} .

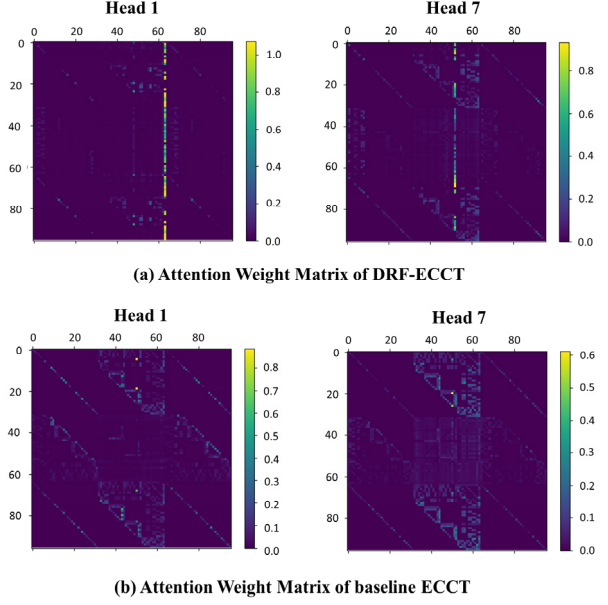


Fig. 6. Attention weight matrices during the inference stage at 4dB - baseline ECCT [10] vs DRF-ECCT. (a) Attention weight matrices for DRF-ECCT. (b) Attention weight matrices for baseline ECCT [10].

For inference, the systematic generator matrix $\mathbf{G}$ is used for generation of codewords through Galois field matrix product, defined by $\mathbf{x} = \mathbf{s}\mathbf{G}$. With this operation, 128 samples are generated until the tested number of error frame reaches 100 error frames to obtain accurate BER results. Similarly, Monte-Carlo simulations are conducted for BP (50 iterations) with up to 10^8 samples to ensure accurate results across all E_b/N_0 regions. All methods are tested over the E_b/N_0 range between 1dB and 7dB.

B. Decoding performance of proposed methods

Fig. 4 shows the logarithmic BER for LDPC(121,80) (Fig. 4a), Polar(64,32) (Fig. 4b) and BCH(63,51) (Fig. 4c) over the test ranges of E_b/N_0 where the differences between the models are most prominent. As can be observed from the logarithmic BER results in Fig. 4, the performance superiority of DRF-ECCT, are prominent for all code types and different coding rate. Without layer reuse, the proposed model shows improvements over baseline ECCT in operational E_b/N_0 region (4dB - 6dB), with 0.15dB performance gains for BCH(63,51), 0.2dB for LDPC(121,80) and Polar(64,32). With the addition of layer reuse, the DRF-ECCT has the best performance, able to achieve performance gains of 0.4dB for LDPC(121,80) and Polar(64,32), and 0.3dB for BCH(63,51).

C. Training convergence

In Fig. 5, the validation convergence of logarithmic BER over 1000 epochs is provided for the baseline ECCT and DRF-ECCT on LDPC (121, 80) during the training stage. From the obtained results, it is proven that DRF-ECCT, either with or without layer reuse, provides the best results. The results in Fig. 5 show that DRF-ECCT models achieve considerably

TABLE II
MODEL SIZE COMPARISON BETWEEN ECCT [10] AND DRF-ECCT WITH LAYER REUSE.

Modules	Number of layers (M / M')	Number of parameters	Size difference (%)
P_{ECCT}	6	1189632	N/A
$P_{DRF-ECCT}$ (without layer reuse)	6	1190400	$\approx$
$P_{DRF-ECCT}$ (with layer reuse)	10	925440	$\downarrow$ 22%

better convergence speed compared to the baseline ECCT. This emphasizes the essentiality of DRF-ECCT in generalization, as both validation during training and inference are consistent.

D. Effective high-reliability attention with DRF

In this subsection, the effect of the proposed DRF mechanism is analyzed for our tested Polar(64,32). Fig. 6 demonstrates examples of attention weight matrices for DRF and baseline ECCTs during inference. From the attention weights of the trained DRF-ECCT (Fig. 6a) and baseline ECCT (Fig. 6b), it is certain that DRF provides a better concentrated attention outputs than the baseline ECCT. The attention weight matrices of the baseline ECCT shows fewer highlighted positions with sparse and disconnected spatial distribution in the attention matrix. On the contrary, the attention outputs of the DRF-ECCT are significantly more concentrated with a bright highlighted column at the position of the high-magnitude position that stand out while other positions are reduced close to zero.

E. Efficiency of the layer reuse strategy

The layer reuse strategy enables improvements in both decoding performance and parameter utilization efficiency. As shown in Fig. 4, the DRF-ECCT model with layer reuse performs better than both the baseline ECCT and the basic DRF-ECCT. In Tab. II, the actual number of parameters for ECCT and DRF-ECCT with layer reuse is presented, which shows the reduction by 22% of the DRF-ECCT compared to the baseline ECCT [10].

V. CONCLUSION

In the rise of transformer-based designs for communication problems such as ECC decoding, DRF-ECCT is proposed to bring additional compatibility between reliability decoding and ECCT as well as improving its overall performance. The evaluation of DRF-ECCT is conducted based on various aspects, including comparison between the proposed methods and baseline methods using BER over operating E_b/N_0 range for different code types, analysis of the effect of DRF for attention weight matrix. In decoding comparison, DRF-ECCT achieves best performance, resulting in improvements between 0.2dB and 0.4dB compared to baseline ECCT. The layer reuse is introduced to further improve model efficiency, which provides additional performance gains. In the analysis of DRF attention matrix, the high-reliability filtering mechanism of

DRF is verified. However, similar to the baseline ECCT [10], DRF-ECCT requires learning each code structure, coding rate, and code length separately, which amounts to considerably extensive training time to cover variety of codes. In both cases, it is necessary for a design allowing heterogeneous learning of different code structures within a certain range of code length.

REFERENCES

- [1] F. A. Aoudia and J. Hoydis, "Model-free training of end-to-end communication systems," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 11, pp. 2503-2516, Nov. 2019.
- [2] T. O'Shea and J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Trans. Cogn. Commun. Netw.*, vol. 3, no. 4, pp. 563-575, Dec. 2017.
- [3] D. García *et al.*, "Model-free machine learning of wireless SISO/MIMO communications," *Comp. Commun.*, vol. 181, pp. 192-202, Jan. 2022.
- [4] J. Clausius *et al.*, "Serial vs. parallel turbo-autoencoders and accelerated training for learned channel codes," in *Proc. 11th Inter. Symp. Topics Coding (ISTC)*, 2021, pp. 1-5.
- [5] A. Buchberger, C. Häger, H. D. Pfister, L. Schmalen and A. Graell i Amat, "Pruning and quantizing neural belief propagation decoders," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 1957-1966, July 2021.
- [6] G. Larue, L. -A. Dufrene, Q. Lampin, H. Ghauch, and G.R.B. Othman, "Neural belief propagation auto-encoder for linear block code design," *IEEE Trans. Commun.*, vol. 70, no. 11, pp. 7250-7264, Nov. 2022.
- [7] D. Liu, M. Bober, and J. Kittler, "Neural belief propagation for scene graph generation," *IEEE Trans. Pattern Anal. Mach. Intel.*, vol. 45, no. 8, pp. 10161-10172, Aug. 2023.
- [8] Q. Wang *et al.*, "Normalized Min-Sum neural network for LDPC decoding," *IEEE Trans. Cogn. Commun. Netw.*, vol. 9, no. 1, pp. 70-81, Feb. 2023.
- [9] F. Liang, C. Shen, and F. Wu, "An iterative BP-CNN architecture for channel decoding," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 144-159, Feb. 2018.
- [10] Y. Choukroun and L. Wolf, "Error correction code transformer," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp.38695-38705.
- [11] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol.30, 2017.
- [12] A. Bennatan, Y. Choukroun, and P. Kisilev, "Deep learning for decoding of linear codes - a syndrome-based approach," in *Proc. 2018 IEEE Inter. Symp. Info. Theory (ISIT)*, Vail, CO, USA, 2018, pp. 1595-1599.
- [13] T. Richardson and R. Urbanke, "The capacity of low-density parity check codes under message-passing decoding," *IEEE Trans. Inf. Theory*, vol. 47, pp. 599-618, Feb. 2001.
- [14] J. Devlin *et al.*, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv preprint arXiv:1810.04805*.
- [15] K. Han *et al.*, "A survey on vision transformer," *IEEE Trans. Pattern Anal. Mach. Intel.*, vol. 45, no. 1, pp. 87-110, Jan. 2023.
- [16] Y. Shao, E. Ozfatura, A. G. Perotti, B. M. Popović, and D. Gündüz, "AttentionCode: Ultra-reliable feedback codes for short-packet communications," *IEEE Trans. Commun.*, vol. 71, no. 8, pp. 4437-4452, Aug. 2023.
- [17] S. Yao, K. Niu, S. Wang and J. Dai, "Semantic coding for text transmission: an iterative design," *IEEE Trans. Cogn. Commun. Netw.*, vol. 8, no. 4, pp. 1594-1603, Dec. 2022.
- [18] D. Hendrycks and K. Gimpel, "Gaussian error linear units (gelus)," 2016, *arXiv preprint arXiv:1606.08415*.
- [19] M. Helmling *et al.*, "Database of channel codes and ML simulation results", www.uni-kl.de/channel-codes, 2019.
- [20] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv preprint arXiv:1412.6980*.
- [21] J. Pearl, *Probabilistic reasoning in intelligent systems: Networks of plausible inference*. Morgan kaufmann, 1988.
- [22] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," 2016, *arXiv preprint arXiv:1608.03983*.